VERIQTA

50 Production Incidents

Every DevOps Engineer Should Know



REAL-WORLD INCIDENTS | ROOT CAUSE ANALYSIS | TROUBLESHOOTING STEPS | FIXES & PREVENTION | SENIOR PROJECTS

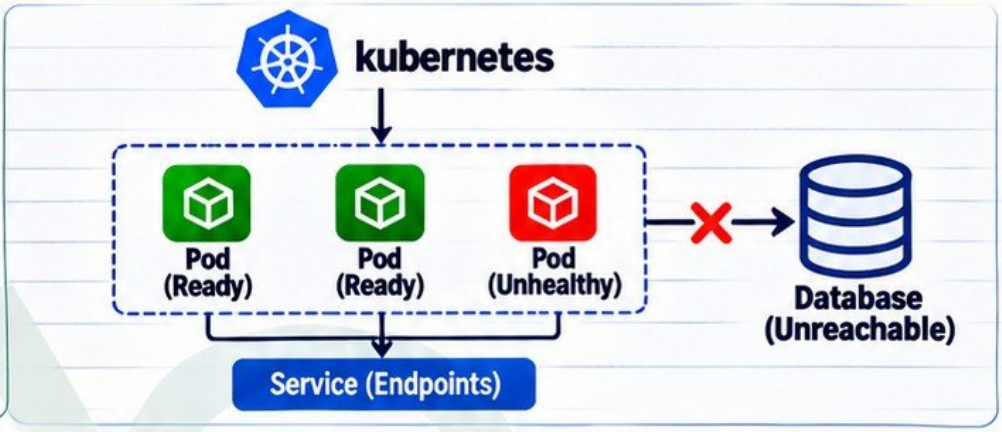
@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

The Deployment Succeeded, but Production Is Failing

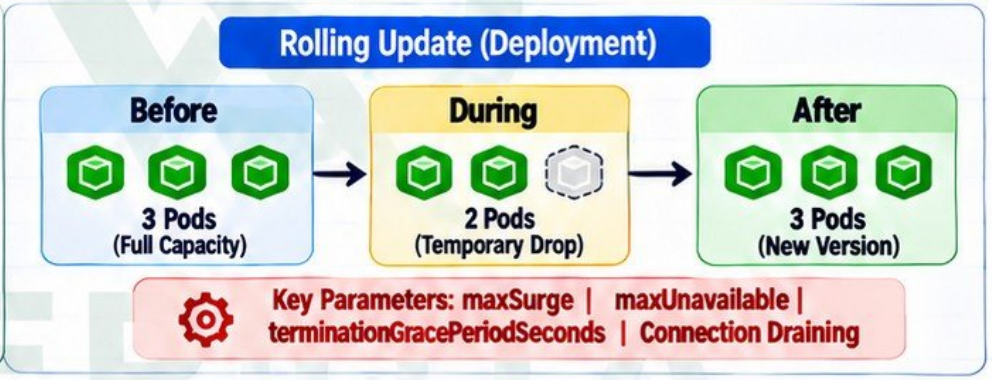
Incident 01 — The Healthy Kubernetes Pod That Cannot Serve Real Traffic

- Readiness checks return HTTP 200 while the application cannot reach its database.
- Investigate endpoint membership, dependency failures, application traces, and user-facing 5xx errors.
- Lab: Build a Kubernetes service with a misleading readiness probe, then correct its health contract.



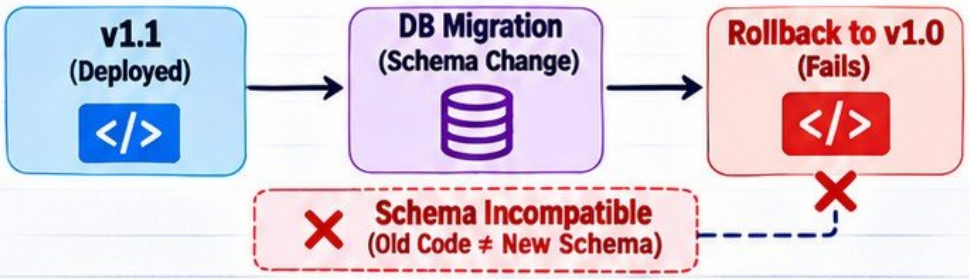
Incident 02 — The Rolling Deployment That Removed Too Much Capacity

- A deployment creates a temporary capacity shortage despite a successful rollout.
- Examine maxSurge, maxUnavailable, termination grace periods, and in-flight requests.
- Lab: Reproduce dropped requests during rolling updates and implement connection draining.



Incident 03 — The Rollback That Restored the Code but Not The Service

- The old application version cannot operate against a newly migrated database schema.
- Investigate migration compatibility, rollback sequencing, and data recovery risks.
- Lab: Implement expand-and-contract migrations and test safe rollback.

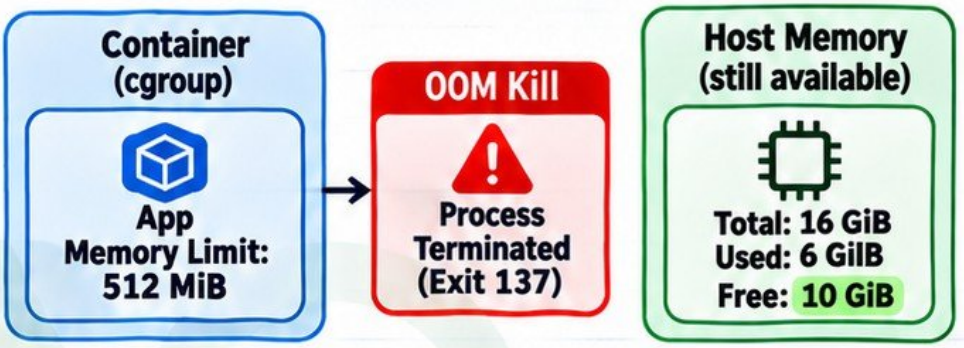


Senior Project: Build a release pipeline that verifies availability during deployment and rollback, not just deployment completion.

Linux Failures That Masquerade as Application Bugs

Incident 04 — The Server With Free Memory That Keeps Killing Processes

- Linux cgroup limits trigger OOM kills despite available host memory.
- Investigate memory.events, kernel logs, container limits, and memory working sets.
- Lab: Reproduce a cgroup OOM and distinguish it from host-wide memory exhaustion.




```
$ cat /sys/fs/cgroup/memory.events
low      0
high     0
max      1
oom      1
oom_kill 1
```

```
$ dmesg | grep -i oom
[ 8452.34] Memory cgroup out of memory:
Killed process 3287 (app) total-vm:842M
RSS:524M, container: app-cgroup
```

Incident 05 — The Disk With Free Space That Cannot Create Files

- Inode exhaustion stops writes while disk capacity appears available.
- Investigate df -h, df -i, directory growth, and file creation failures.
- Lab: Generate millions of tiny files in a disposable filesystem and recover safely.

```
$ df -h
Filesystem      Size  Used Avail Use%
/dev/xvda1      100G   20G   80G  20%
```

```
$ df -i /data
Filesystem      Inodes IUsed IFree IUse%
/dev/xvda1      6.5M   6.5M  0    100%
```



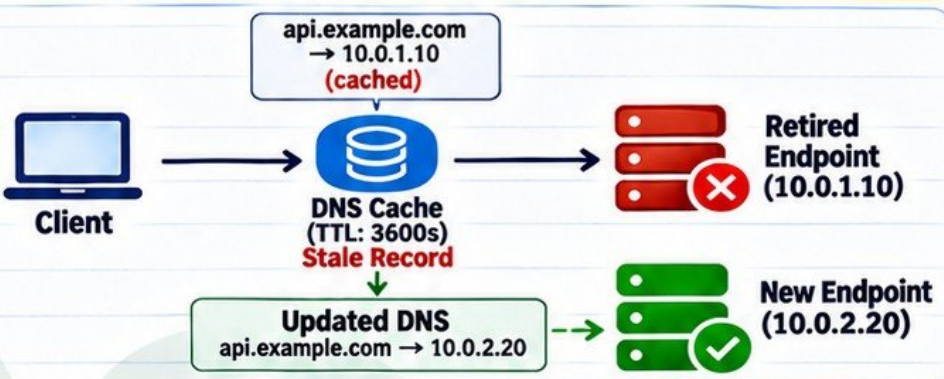
Senior Project: Build a Linux failure-classification toolkit that distinguishes CPU, memory, disk blocks, inodes, and process limits.

VERIQTA

The Network Is Reachable, but the Application Is Unavailable

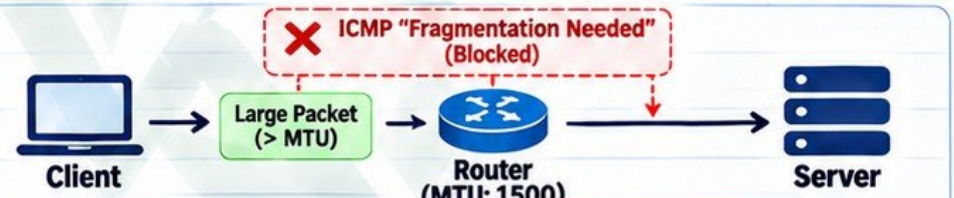
Incident 06 — The DNS Record That Outlived Its Infrastructure

- A stale resolver cache continues sending traffic to retired endpoints.
- Examine DNS TTLs, caching layers, negative caching, and service discovery.
- Lab: Reproduce inconsistent resolution during a simulated service migration.



Incident 07 — The TCP Connection That Succeeds but Cannot Transfer Data

- An MTU or Path MTU Discovery problem causes large requests to stall.
- Investigate packet captures, retransmissions, MSS, ICMP filtering, and fragmentation.
- Lab: Simulate an MTU black hole with Linux network namespaces.



Key Checks

- tcpdump – capture and inspect packets
- tracepath – test path MTU
- ss – verify connection state
- ip link – check MTU settings

Likely Cause

- MTU mismatch
- Blocked ICMP (PMTUD)
- Packets dropped / not fragmented

Incident 08 — The Firewall Rule That Broke Only Long-Lived Connections

- Stateful connection tracking or idle timeouts interrupt WebSockets and persistent sessions.
- Examine conntrack state, keepalives, connection lifetimes, and proxy timeouts.
- Lab: Reproduce intermittent session disconnects and identify the failing network layer.



Investigation Commands

- conntrack -L – view connection state
- ss -tnp – check active sessions
- tcpdump – verify keepalive traffic
- grep timeout (proxy logs)

Common Fixes

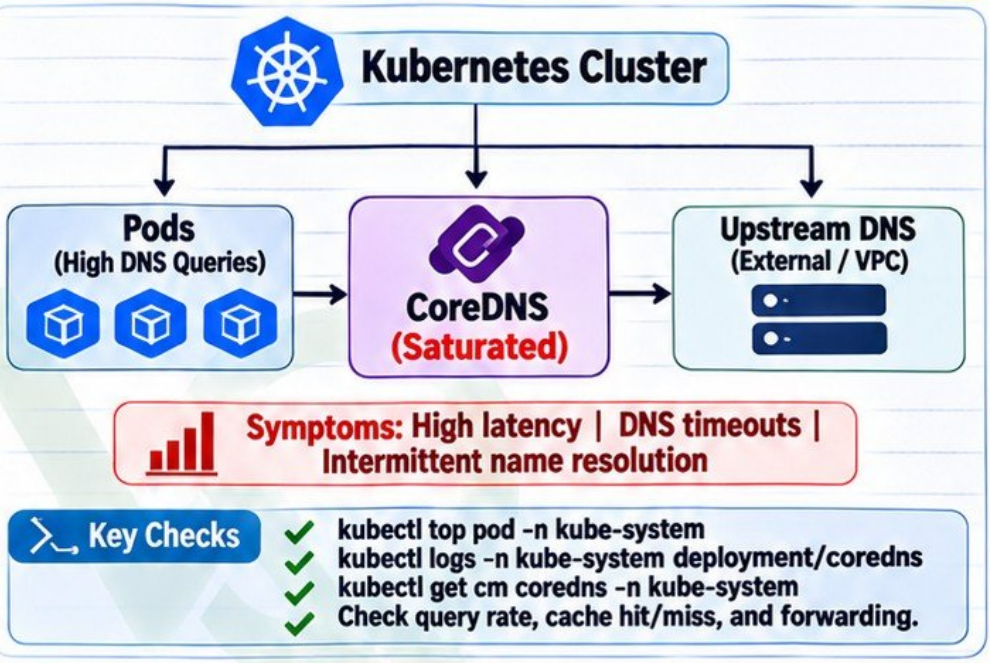
- Increase idle timeout
- Enable TCP keepalive / WebSocket ping
- Align proxy and firewall timeouts
- Tune conntrack limits if required

Senior Project: Build a network incident laboratory using Linux namespaces, tcpdump, ss, dig, and conntrack.

Kubernetes Networking That Fails Without a Pod Crash

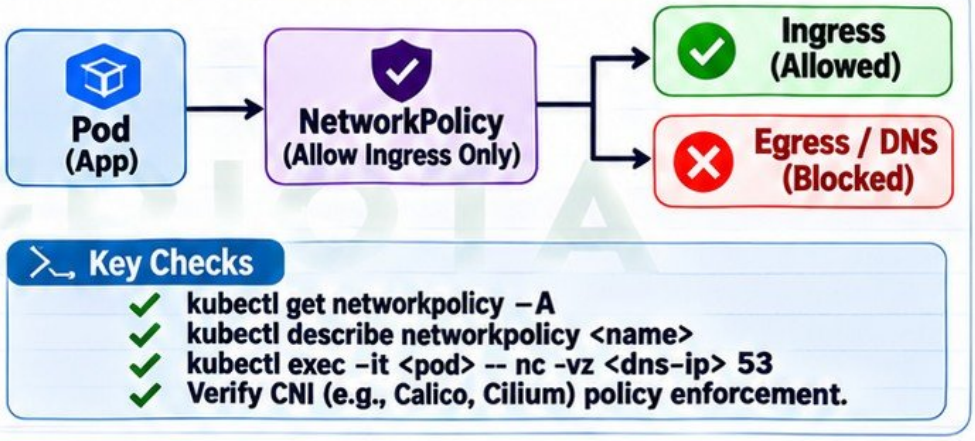
Incident 09 — The Cluster DNS Service That Became the Bottleneck

- CoreDNS saturation causes application-wide latency and intermittent resolution failures.
- Investigate query rates, cache misses, DNS forwarding, and resolver behavior.
- Lab: Generate DNS-heavy workloads and measure their effect on service latency.



Incident 10 — The NetworkPolicy That Passed Testing but Blocked Production

- A policy permits ingress but unexpectedly blocks required egress or DNS traffic.
- Examine pod selectors, namespace selectors, CNI enforcement, and effective policies.
- Lab: Deploy restrictive policies and troubleshoot missing connectivity.



Senior Project: Create a Kubernetes connectivity diagnostic workflow that identifies DNS, policy, service, endpoint, and CNI failures.

The Kubernetes Control Plane That Cannot Keep Up

Incident 11 — The etcd Database That Turned Cluster Operations Into Timeouts

- Storage latency or excessive object churn slows Kubernetes API operations.
- Investigate etcd request latency, database size, compaction, and disk performance.
- Lab: Use an isolated disposable cluster to observe API degradation under controlled load.



Key Checks

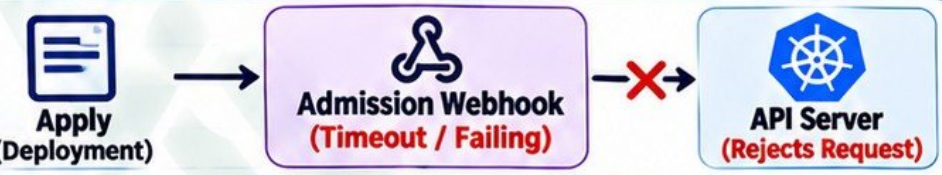
- etcdctl endpoint status
- etcdctl endpoint health
- etcdctl defrag
- Check disk IOPS and latency

Common Symptoms

- kubectl commands time out
- High etcd request latency
- Large database size / no compaction
- Slow API responses

Incident 12 — The Admission Webhook That Blocked Every Deployment

- A failing webhook prevents new workloads from being admitted.
- Examine webhook timeouts, failurePolicy, certificate validity, and service availability.
- Lab: Break a test admission webhook, recover deployment capability, and design a safe failure policy.



Key Checks

- kubectl get validatingwebhookconfigurations
- kubectl describe <webhook>
- Check webhook service and pod status
- Verify certificate (TLS) and timeouts

Fix / Design

- Set appropriate failurePolicy (e.g., Ignore for non-critical)
- Ensure webhook high availability
- Use timeout and health checks
- Monitor webhook latency

Incident 13 — The Autoscaler That Added Nodes Too Late

- Pod demand rises faster than nodes become usable.
- Investigate scheduling events, node startup time, quotas, subnet IP capacity, and scaling signals.
- Lab: Simulate a traffic spike and measure time to usable capacity.



Key Checks

- kubectl describe pod (scheduling events)
- kubectl get nodes
- Check autoscaler logs and events
- Verify quotas, subnet IPs, and launch time

Possible Causes

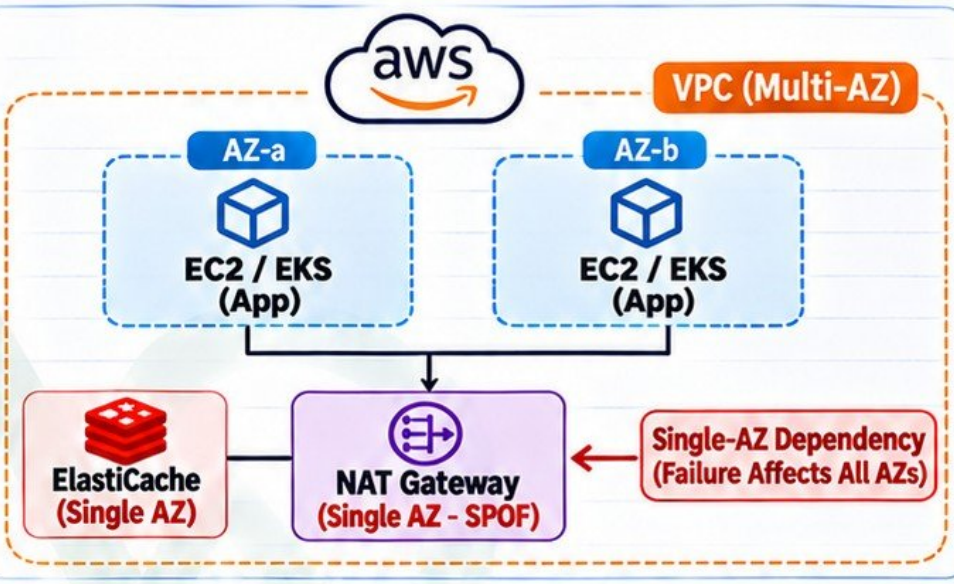
- Node launch time is too long
- Insufficient IPs or quota limits
- Slow instance provisioning / images
- Autoscaler thresholds not tuned

Senior Project: Design a cluster-control-plane resilience assessment with recovery procedures and evidence-based thresholds.

AWS Infrastructure That Fails Across Availability Zones

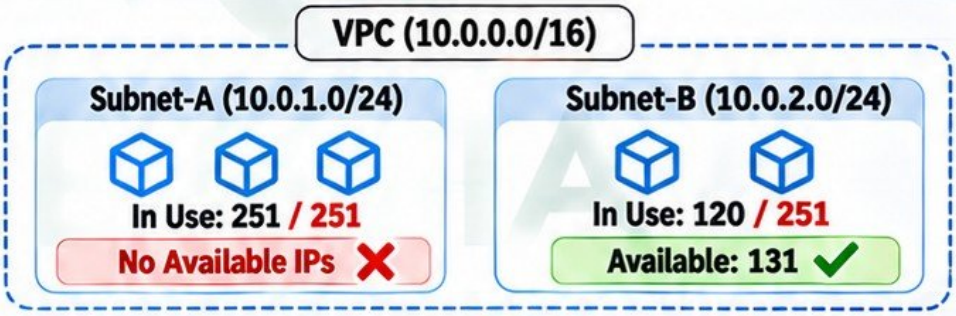
Incident 14 — The Multi-AZ Application With a Single-AZ Dependency

- Compute spans multiple AZs, but NAT, caching, or another critical dependency remains concentrated.
- Map actual request dependencies and identify hidden shared failure domains.
- Lab: Simulate an AZ dependency failure and measure service impact.



Incident 15 — The Subnet That Ran Out of IP Addresses During Scaling

- Instances or pods cannot launch because subnet address capacity is exhausted.
- Investigate available IP counts, ENI allocation, VPC CNI settings, and scheduling events.
- Lab: Model subnet consumption and validate scale-out headroom.




```
$ kubectl describe pod
```

FailedScheduling: 0/10 nodes are available: pod has unbound immediate PersistentVolumeClaims, **Insufficient pods IPs in subnet.**

Key Checks

- ✓ Check subnet CIDR and available IPs
- ✓ Review ENI and IP allocation (VPC CNI)
- ✓ Inspect scheduling events and errors
- ✓ Validate scale-out IP headroom

Senior Project: Build an AWS failure-domain analyzer that produces an AZ dependency map and recovery recommendations.

The Database Is Running, but the Business Is Down

Incident 16 The Connection Pool That Exhausted the Database

- Application replicas scale out and collectively exceed database connection limits.
- Investigate pool sizing, connection churn, wait times, and database session counts.
- Lab: Reproduce connection exhaustion and introduce bounded pooling.



Key Checks

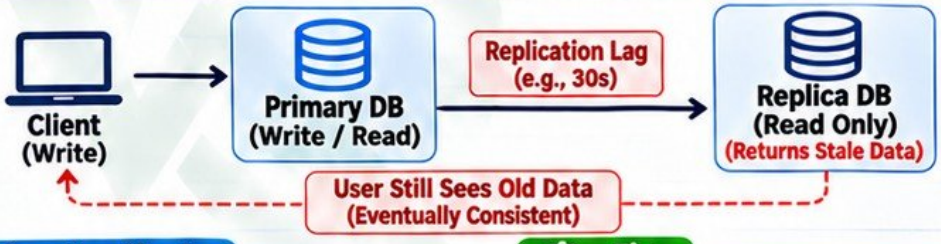
- SHOW max_connections;
- SHOW PROCESSLIST;
- Check application pool size and idle timeout
- Monitor connection count and wait time

Fix

- Set a max pool size (e.g., 50 per pod)
- Enable connection reuse
- Add pooling (PgBouncer/HikariCP)
- Monitor and alert on connection usage

Incident 17 The Replica That Returned Yesterday's Answer

- Replication lag causes stale reads after successful writes.
- Examine replica delay, read-after-write requirements, and routing policies.
- Lab: Simulate delayed replication and enforce consistency for critical operations.



Key Checks

- Check replication lag (e.g., SHOW REPLICA STATUS;)
- Verify read routing (primary vs replica)
- Confirm read-after-write requirements

Fix

- Route critical reads to primary
- Reduce replication lag
- Use read-after-write consistency or session stickiness

Incident 18 The Schema Migration That Locked a Busy Table

- A seemingly harmless database change blocks production transactions.
- Investigate lock queues, long-running transactions, migration strategies, and rollback options.
- Lab: Reproduce lock contention and implement a safer migration process.



Key Checks

- Check active locks (e.g., pg_locks)
- Identify long-running transactions
- Review migration plan and lock type
- Check impacted queries and duration

Fix / Best Practice

- Use online/zero-downtime migrations (e.g., additive changes, backfill, then switch)
- Set lock and statement timeouts
- Run migrations during low traffic
- Have a tested rollback plan

Senior Project: Build a database incident response dashboard combining connection saturation, locks, replication lag, and query latency.

VERIQTA

When Reliability Features Cause Cascading Failures

Incident 19 — The Retry Storm That Prevented Recovery

- Multiple service layers retry the same failed request and amplify traffic.
- Model retry multiplication, backoff, jitter, and request deadlines.
- Lab: Reproduce a retry storm and implement retry budgets.



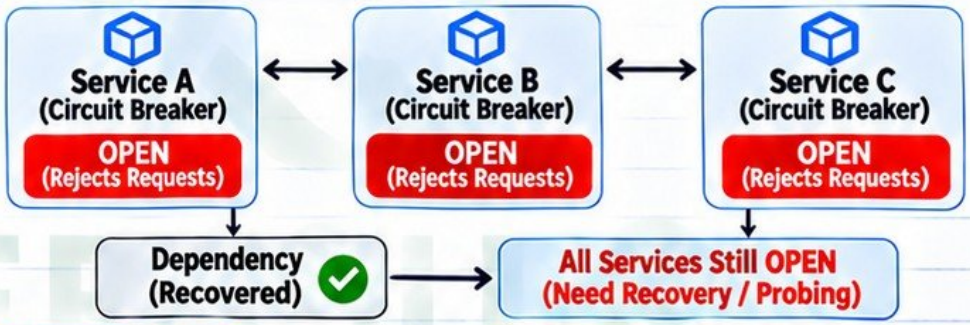
Retry Amplification:
1 Request → 3 → 9 → 27 Requests

Key Checks
- Trace request retries across services
- Check retry count and backoff policy
- Verify request deadlines / timeouts
- Measure traffic amplification rate

Fix / Best Practice
- Use exponential backoff + jitter
- Set retry budgets (max retries)
- Add request timeouts and circuit breaker
- Remove unnecessary retries

Incident 20 — The Circuit Breaker That Opened Across Every Service

- Correlated circuit breakers reject requests after the underlying dependency recovers.
- Investigate breaker thresholds, recovery probes, half-open states, and dependency isolation.
- Lab: Compare synchronized and staggered circuit-breaker recovery.



Key Checks
- Check circuit breaker state (open/half-open)
- Review failure thresholds and window
- Verify recovery probes and success rate
- Check if breakers are correlated / cascading

Fix / Best Practice
- Use independent thresholds per service
- Configure half-open with safe probing
- Stagger recovery (jittered re-enabling)
- Isolate dependencies and limit blast radius

Senior Project: Build a microservice resilience simulator that measures retry amplification, circuit-breaker behavior, and recovery time.

AWS and Google document how retries, resource leaks, and overload can turn localized failures into cascading outages.

The Message Queue That Became a Production Incident

Incident 21 — The Consumer That Processed the Same Payment Twice

- A worker completes a transaction but fails before acknowledging the message.
- Investigate delivery guarantees, ack timing, idempotency keys, and deduplication.
- Lab: Simulate consumer crashes and prevent duplicate financial operations.



Key Checks

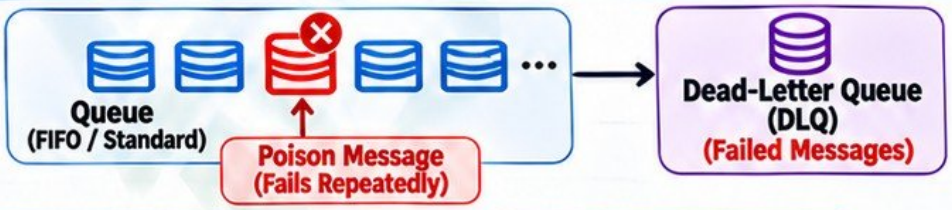
- Check delivery semantics (at-least/exactly-once)
- Review ACK (acknowledgment) timing
- Verify idempotency key implementation
- Check for duplicates and transaction safeguards

Fix / Best Practice

- Use idempotency keys (e.g., orderId)
- Make consumers idempotent
- ACK only after a successful commit
- Add deduplication at the application/DB layer

Incident 22 — The Poison Message That Blocked an Entire Queue

- One invalid message repeatedly fails and delays other work.
- Examine visibility timeouts, retry policies, dead-letter queues, and replay controls.
- Lab: Build a poison-message recovery workflow without losing valid messages.



Key Checks

- Check error logs and failure reason
- Review visibility timeout and retry count
- Confirm DLQ (dead-letter queue) configuration
- Validate message replay and reprocessing flow

Fix / Best Practice

- Move failing messages to a DLQ
- Set a sensible retry policy and backoff
- Implement message validation before process
- Provide a safe replay mechanism after fix

Incident 23 — The Queue That Looked Healthy While Customer Orders Aged

- Message count stays manageable while the age of the oldest message grows.
- Investigate processing throughput, backlog age, consumer saturation, and business deadlines.
- Lab: Build backlog alerts based on processing delay rather than queue depth.



Key Checks

- Check oldest message age (not just count)
- Measure consumer processing rate (msgs/sec)
- Inspect worker health and resource usage
- Verify if SLA / business deadlines are at risk

Fix / Best Practice

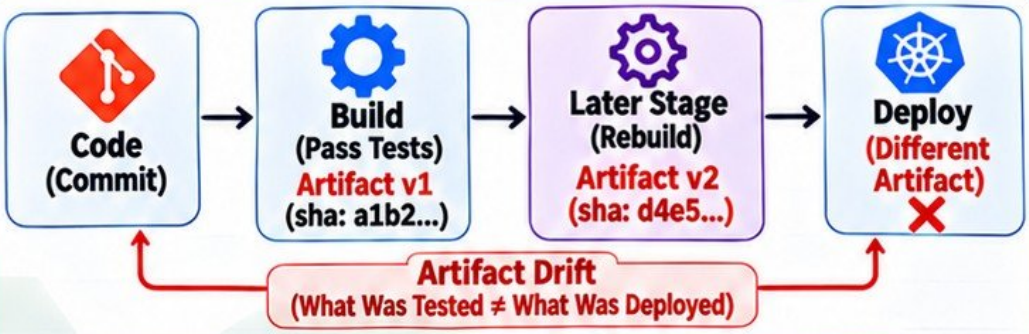
- Add alerts on message age (e.g., > 5 min)
- Scale consumers when processing slows
- Fix bottlenecks (CPU, DB, external API)
- Define clear SLAs and backlog thresholds

Senior Project: Build an order-processing platform with SQS or RabbitMQ, idempotent consumers, dead-letter handling, and controlled replay.

CI/CD Pipelines That Become Production Attack Paths

Incident 24 The Pipeline That Deployed an Untested Artifact

- A build passes tests, but a later stage rebuilds a different artifact.
- Investigate artifact immutability, digests, provenance, and promotion controls.
- Lab: Demonstrate artifact drift and enforce digest-based deployment.



Key Checks

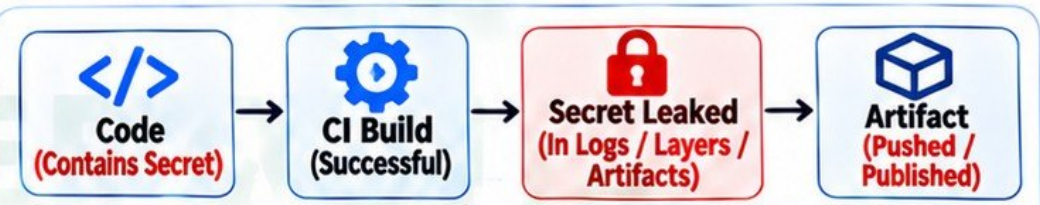
- Compare artifact digests (e.g., sha256)
- Verify the same artifact is promoted
- Check build and deployment logs
- Validate provenance (who built, when, from what)

Fix / Best Practice

- Use immutable, versioned artifacts
- Deploy by digest (not by tag)
- Implement artifact promotion (dev → staging → prod)
- Use provenance and artifact signing (e.g., Sigstore / cosign)

Incident 25 The Secret That Escaped Through a Successful Build

- A credential appears in CI logs, build layers, or uploaded artifacts.
- Examine masking limitations, credential scope, secret scanning, and short-lived identity.
- Lab: Use synthetic credentials to test detection, containment, and rotation.



Key Checks

- Scan code, logs, and build artifacts for secrets
- Review masking and redaction behavior
- Check credential scope and permissions
- Identify and rotate any exposed credentials

Fix / Best Practice

- Use a secrets manager (e.g., AWS Secrets Manager, GCP Secret Manager)
- Avoid long-lived credentials
- Mask and block secrets in CI logs
- Use short-lived, least-privilege identities (e.g., OIDC / workload identity)
- Add automated secret scanning (e.g., TruffleHog, Gitleaks)

Senior Project: Build a secure delivery pipeline with immutable artifacts, signed provenance, temporary credentials, and deployment verification.

Terraform Changes That Destroy More Than Intended

Incident 26 — The Terraform Plan That Replaced a Critical Resource

- A seemingly minor configuration change requires replacement of a production database or network resource.
- Investigate provider schemas, state, resource identity, and destructive plan actions.
- Lab: Detect replacements in a disposable infrastructure environment before apply.



Key Checks

- Review plan output for replace (-/+)
- Understand why replacement is required (schema / immutable fields)
- Check resource identity and state mapping
- Test changes in a non-production environment

Fix / Best Practice

- Use terraform plan before apply
- Enable plan approval (manual gate)
- Use lifecycle rules (e.g., prevent_destroy)
- Test changes in staging / disposable env
- Review provider release notes and schema changes

Incident 27 — The State File That Became a Security Incident

- Terraform state exposes sensitive configuration or credentials.
- Examine state access, backend encryption, audit logs, and secret handling.
- Lab: Implement least-privilege remote state access and automated access verification.



Key Checks

- Check what sensitive data is in the state
- Verify state backend (e.g., S3, GCS, Azure)
- Confirm encryption (at rest and in transit)
- Review access policy and audit logs
- Ensure no secrets are hardcoded in code

Fix / Best Practice

- Use a remote backend with encryption (e.g., S3 + KMS, GCS, Azure Storage)
- Apply least-privilege access (read/write as needed)
- Enable audit logging and access alerts
- Use secret managers (e.g., AWS Secrets Manager)
- Avoid storing plaintext secrets in state

Incident 28 — The Emergency Console Fix That Terraform Reversed

- Manual incident mitigation creates drift that a later pipeline removes.
- Investigate drift detection, incident change control, and reconciliation.
- Lab: Introduce controlled drift and design a safe recovery process.



Key Checks

- Detect drift (terraform plan / drift tools)
- Compare actual vs. desired state
- Validate if manual change was intended/temporary
- Review change control and incident records
- Plan a safe reconciliation approach

Fix / Best Practice

- Use drift detection (e.g., terraform plan in CI)
- Record emergency changes and reasons
- Reconcile changes through code, not manual fixes
- Use change windows and approvals
- Consider ignore_changes for intentional temporary drift

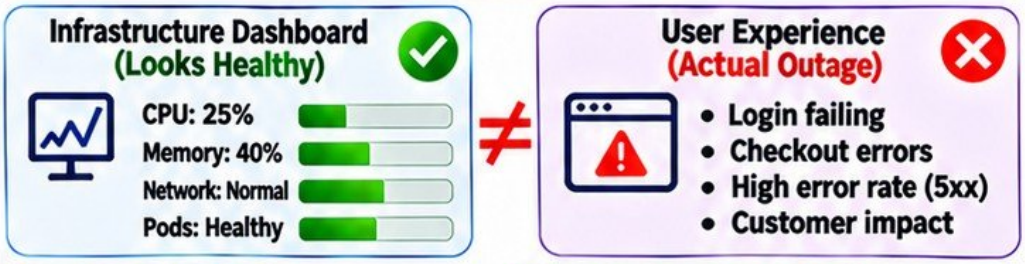
Senior Project: Build a Terraform production-safety gate that checks destructive changes, drift, permissions, and recovery readiness.

VERIQTA

Observability That Misleads the On-Call Engineer

Incident 29 — The Green Dashboard During a Customer-Facing Outage

- Infrastructure metrics appear normal while real user transactions fail.
- Investigate synthetic transactions, SLIs, dependency health, and business outcomes.
- Lab: Create an outage invisible to CPU and memory monitoring but visible to user-journey checks.



Key Checks

- Run synthetic user journeys (e.g., login → add to cart → checkout)
- Review SLIs (latency, success rate, error rate)
- Check dependency health (DB, API, external)
- Correlate metrics with business outcomes (e.g., orders, payments, signups)

Fix / Best Practice

- Add synthetic monitoring and real-user monitoring (RUM)
- Define and track SLIs/SLOs (availability, latency, success rate)
- Monitor critical dependencies and latency
- Use business-level alerts (e.g., failed orders, revenue drop)

Incident 30 — The Log Pipeline That Caused the Application Outage

- Synchronous logging or blocked log shipping exhausts application resources.
- Examine logging backpressure, buffer limits, disk consumption, and collector failures.
- Lab: Simulate a failed log destination and validate that application traffic survives.



Key Checks

- Verify logging mode (synchronous vs. async)
- Check log queue/buffer size and drop policy
- Inspect disk usage and log file growth
- Review collector/forwarder health and errors
- Correlate resource usage with log pipeline issues

Fix / Best Practice

- Use asynchronous, non-blocking logging
- Set buffer limits and safe drop/retention policy
- Add storage quotas and log rotation
- Make log pipeline highly available (retries, HA)
- Ensure application remains available when log destination fails

Senior Project: Build an observability system with OpenTelemetry, Prometheus, Grafana, and application-level SLOs.

OpenTelemetry | Prometheus | Grafana | SLOs

Security Controls That Accidentally Disable Production

Incident 31 — The Certificate Rotation That Broke Service Authentication

- A certificate rotates successfully, but clients reject the new trust chain.
- Investigate CA bundles, trust distribution, hostname verification, and mTLS.
- Lab: Simulate certificate rotation failures and recover without disabling verification.



Common Causes
- Missing CA bundle
- Incomplete trust chain
- Hostname mismatch
- mTLS misconfiguration

Key Checks
- Validate certificate chain (CA → leaf)
- Confirm trust store / CA bundle updates
- Verify hostname / SAN correctness
- Check mTLS client and server config
- Inspect client errors (TLS / x509)

Fix / Best Practice
- Distribute trust bundle before rotation
- Use overlapping (dual) certificates
- Automate and test rotation in staging
- Monitor TLS errors and client success rate
- Keep verification enabled (do not bypass)

Incident 32 — The IAM Policy That Worked Until the Next Deployment

- A policy change removes a permission needed only during startup, scaling, or recovery.
- Investigate CloudTrail, assumed roles, permission boundaries, and least-privilege testing.
- Lab: Reproduce an IAM permission regression and implement permission validation.



Investigate With
- CloudTrail logs
- Assumed role details
- Permission boundaries
- Required actions (e.g., ECR, S3, KMS, ECS/EKS)

Key Checks
- Review recent IAM policy changes
- Check CloudTrail for AccessDenied errors
- Validate required permissions at startup/scale
- Test permissions in a staging environment
- Confirm role, policy, and boundary inheritance

Fix / Best Practice
- Apply least-privilege with lifecycle needs (startup, scaling, recovery)
- Use policy testing (e.g., iam:SimulatePrincipalPolicy)
- Keep deployment pre-checks for critical permissions
- Version and review IAM changes
- Add rollback for permission regressions

Incident 33 — The WAF Rule That Blocked Legitimate Customers

- A security rule misclassifies normal traffic and creates a partial outage.
- Examine rule matches, false-positive rates, request samples, and rollback controls.
- Lab: Deploy a test WAF policy, measure false positives, and stage safe enforcement.



Root Causes
- Overly broad rule patterns
- High false-positive rate
- Missing allowlist exceptions
- No staged rollout or rollback

Key Checks
- Review WAF rule (pattern, condition, scope)
- Analyze blocked requests and samples
- Measure false-positive rate and impact
- Check rule priority and interaction
- Verify rollback and version control

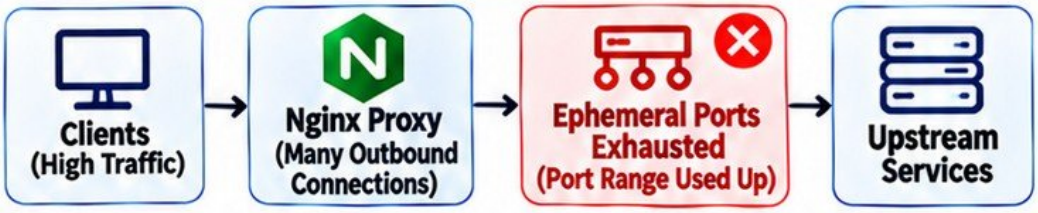
Fix / Best Practice
- Test rules in monitor (count-only) mode first
- Use exception/allowlist for known good traffic
- Roll out gradually (canary / percentage)
- Monitor business impact (errors, conversions)
- Keep a quick rollback procedure

Senior Project: Build a security-change validation pipeline with synthetic traffic, staged rollout, and automatic rollback criteria.

The Hidden Failure Modes of Load Balancers and Proxies

Incident 34 — The Nginx Proxy That Exhausted Ephemeral Ports

- High outbound connection churn exhausts available source-port combinations.
- Investigate connection reuse, TIME_WAIT, upstream pools, and NAT behavior.
- Lab: Reproduce connection churn in a controlled environment and implement connection reuse.



Key Checks

- Check connection count (e.g., ss -s, ss -ant)
- Monitor TIME_WAIT and port usage (e.g., ss -ant state time-wait | wc -l)
- Review Nginx upstream and keepalive settings (keepalive, keepalive_requests)
- Check NAT / SNAT port limits (if applicable)

Fix / Best Practice

- Enable connection reuse (keepalive / upstream keepalive)
- Tune keepalive_timeout and keepalive_requests
- Increase ephemeral port range (e.g., sysctl net.ipv4.ip_local_port_range)
- Scale out or use multiple egress IPs (NAT / load balancer)

Incident 35 — The Load Balancer That Sent Traffic to a Dying Application

- Health checks pass while application workers are blocked or draining.
- Examine health-check semantics, deregistration delays, keepalive behavior, and connection termination.
- Lab: Simulate graceful shutdown failures and eliminate dropped in-flight requests.



Key Checks

- Review health-check configuration (path, interval, timeout, success codes)
- Check deregistration delay (e.g., ALB/NLB)
- Inspect connection states and keepalive (e.g., ss -tan, Nginx/ALB settings)
- Verify application readiness and drain logic (e.g., /ready, /health, preStop hook)

Fix / Best Practice

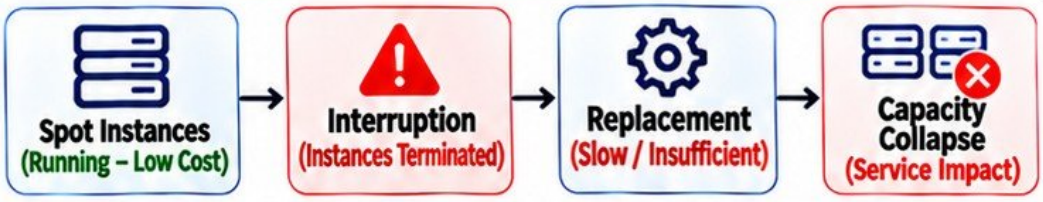
- Use effective readiness and liveness checks (reflect real application state)
- Set an appropriate deregistration delay (e.g., 30–300 seconds, based on workload)
- Enable graceful shutdown (drain + preStop hook)
- Configure keepalive and connection termination correctly
- Test rolling updates and shutdown under load

Senior Project: Build a reverse-proxy resilience lab using Nginx, an application backend, load generation, and packet-level evidence.

The Cloud Cost Decision That Created a Reliability Incident

Incident 36 — The Spot Instance Interruption That Triggered a Capacity Collapse

- Spot instances are interrupted and remove critical capacity.
- Replacements cannot start quickly enough to handle the load.
- Examine interruption handling, instance diversification, workload priority, and spare capacity.
- Lab: Simulate a spot interruption and test workload continuity.



Key Checks

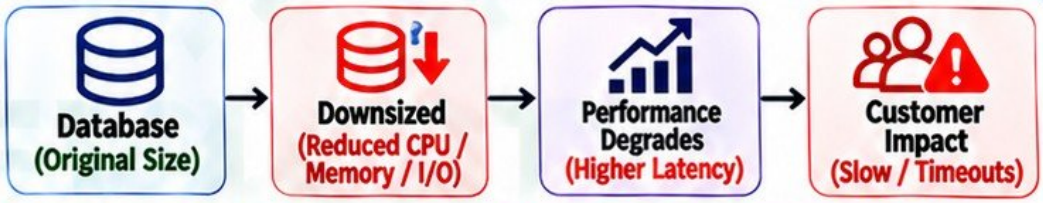
- Review interruption events (e.g., EC2 Spot / AWS Events)
- Validate replacement time and success rate
- Check instance diversification (on-demand, spot, reserved)
- Confirm workload priority and scaling policy
- Verify spare capacity / buffer (N+1, N+2)

Fix / Best Practice

- Handle spot interruptions (e.g., EC2 Spot interruption notice / drain)
- Use a mixed capacity strategy (spot + on-demand)
- Prioritize critical workloads (e.g., separate node groups)
- Maintain spare capacity buffers (e.g., 20–30%)
- Test interruption and recovery regularly (chaos / game day)

Incident 37 — The Database Downsizing That Created a Latency Crisis

- A cost-saving instance change reduces memory or I/O headroom.
- Investigate cache hit rates, IOPS, throughput, queeing, and p99 latency.
- Lab: Benchmark a database under constrained resources and establish safe capacity limits.



Key Checks

- Compare instance specs (before vs. after)
- Check cache hit rate (e.g., Redis / DB cache)
- Review IOPS, throughput, and queue depth
- Analyze latency (p50, p95, p99) and error rate
- Evaluate resource utilization (CPU, memory, I/O)
- Confirm workload pattern (peak vs. steady)

Fix / Best Practice

- Right-size using real workload metrics (not only cost)
- Maintain sufficient memory and I/O headroom
- Use or tune caching (e.g., Redis, in-memory cache)
- Define SLOs (e.g., p99 latency, error rate)
- Test changes in a staging environment with realistic load
- Use automatic rollback if latency or errors exceed thresholds

Senior Project: Build a cost-versus-reliability evaluation that quantifies savings alongside saturation risk and recovery capacity.

Distributed Systems That Lose Correctness Without Going Down

Incident 38 — The Leader Election That Produced Two Active Writers

- A stale leader continues writing after another node takes ownership.
- Examine leases, fencing tokens, clock assumptions, and storage coordination.
- Lab: Simulate stale leadership and prevent conflicting writes.



Key Checks

- Verify leader lease / TTL and renewal
- Use fencing tokens (monotonic ID) for writes
- Check storage coordination (e.g., etcd, Consul, ZooKeeper)
- Detect and alert on dual writers

Fix / Best Practice

- Use short, renewable leases with fencing
- Ensure only the current leader can write (verify token)
- Use a proven coordination store (etcd/Consul)
- Monitor leadership changes and conflicts
- Test leader failover under network delay

Incident 39 — The Cache Invalidation That Served Incorrect Prices

- Application caches retain outdated values after authoritative data changes.
- Investigate TTLs, event delivery, invalidation ordering, and stale reads.
- Lab: Build a price-update system and test cache correctness under message delays.



Key Checks

- Review cache TTL and refresh strategy
- Verify invalidation event delivery (reliable, ordered)
- Check for stale-read patterns (replicas, CDN, local cache)
- Validate price consistency after updates

Fix / Best Practice

- Use event-driven invalidation (e.g., pub/sub)
- Keep TTLs aligned with business requirements
- Ensure ordered and reliable update delivery
- Use cache versioning or write-through patterns
- Automate consistency tests (before release)

Incident 40 — The Clock Skew That Invalidated Authentication Tokens

- Time differences between systems cause valid requests to fail.
- Examine NTP synchronization, token validity windows, timestamps, and audit ordering.
- Lab: Simulate clock skew in isolated services and implement detection.



Key Checks

- Compare system time across services (NTP)
- Verify token validity window (iat, exp)
- Check timestamp use in auth and audit logs
- Detect and alert on clock drift (e.g., > 30s)

Fix / Best Practice

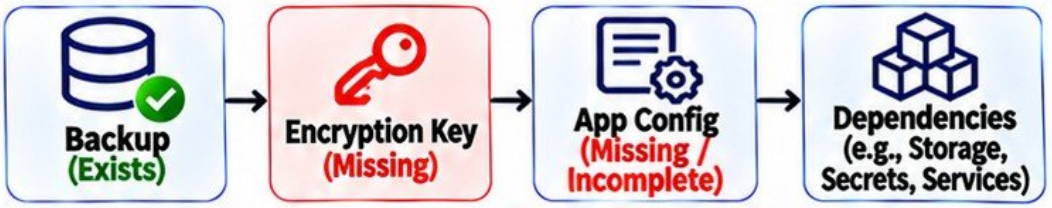
- Use NTP/chrony for time synchronization
- Allow small clock skew (e.g., ±300 seconds)
- Standardize time sources across all services
- Monitor and alert on time drift
- Include time in health checks and deployments

Senior Project: Build a distributed correctness testbed with injected delays, stale leaders, and consistency checks.

Disaster Recovery That Works on Paper but Fails in Practice

Incident 41 — The Backup That Could Not Restore the Application

- Database backups exist, but the restore fails due to missing encryption keys, application configuration, or dependent resources.
- Investigate restore dependencies, backup integrity, and recovery sequencing.
- Lab: Restore a complete application into a clean environment.



Key Checks

- Verify backup integrity (e.g., checksum, test restore)
- Confirm encryption keys are available and valid
- Identify and list all dependencies (config, secrets, data, services)
- Validate restore order and procedure
- Perform a full restore in an isolated environment

Fix / Best Practice

- Back up and secure encryption keys (e.g., KMS / Vault)
- Version and back up application configuration (IaC, config files, secrets)
- Document and test the full recovery sequence
- Run regular restore drills (end-to-end)
- Use automated validation to confirm a recoverable backup

Incident 42 — The Regional Failover That Overloaded the Surviving Region

- Failover routing works, but the destination region lacks sufficient capacity.
- Examine quotas, database capacity, traffic distribution, and dependency limits.
- Lab: Run a controlled regional failover simulation and measure degraded performance.



Key Checks

- Verify regional quotas and limits (compute, database, network)
- Check resource capacity and scaling headroom in the target region
- Measure performance (latency, error rate, throughput) during failover
- Validate traffic distribution and load balancing
- Review dependency limits (e.g., database connections, API quotas)

Fix / Best Practice

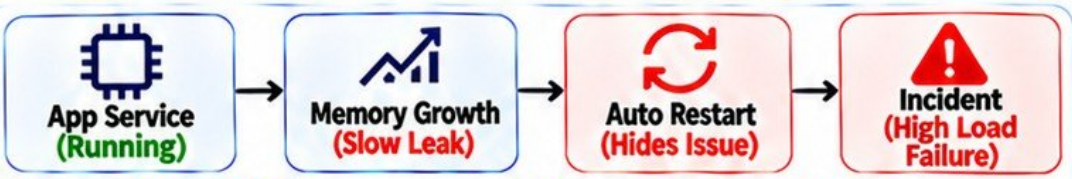
- Maintain pre-provisioned or auto-scalable capacity in the standby region
- Use multi-region or active-standby capacity planning
- Test regional failover regularly with realistic load
- Set and monitor SLOs during failover (latency, error rate, capacity)
- Review and increase critical service quotas in advance

Senior Project: Build a disaster recovery exercise with measured RTO, RPO, recovery dependencies, and documented evidence.

The Production Incident Nobody Detected Early Enough

Incident 43 — The Slow Memory Leak Hidden by Automatic Restarts

- Automatic restarts hide a persistent memory defect until traffic increases.
- Correlate restart frequency, heap growth, workload age, and customer latency.
- Lab: Inject a controlled memory leak and identify it before OOM termination.



Key Checks

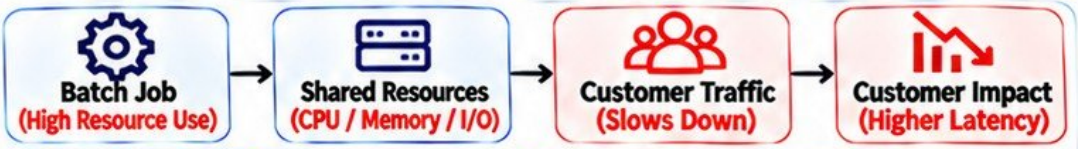
- Track restart frequency and reasons (e.g., OOM, liveness)
- Monitor memory/heap growth trend (e.g., RSS, heap, GC metrics)
- Correlate traffic volume with memory usage
- Detect abnormal memory slope early (e.g., sustained increase)

Fix / Best Practice

- Enable memory profiling and leak detection
- Set early-warning thresholds (trend-based)
- Investigate and fix the memory leak (root cause)
- Use canary or staged rollout after the fix
- Avoid masking issues with automatic restarts

Incident 44 — The Batch Job That Starved Interactive Traffic

- Background processing competes with customer requests for shared resources.
- Examine resource contention, scheduling, concurrency limits, and workload isolation.
- Lab: Reproduce noisy-neighbor behavior and apply resource controls.



Key Checks

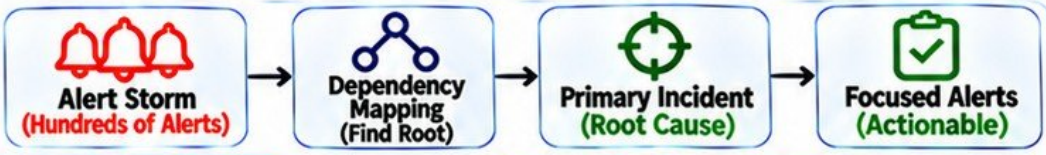
- Measure resource usage by workload (CPU, memory, I/O, disk)
- Identify contention and saturation points (e.g., CPU throttling, high I/O wait)
- Review job scheduling, concurrency, and run windows
- Validate isolation (e.g., cgroups, quotas, separate node pool)

Fix / Best Practice

- Schedule batch jobs during low-traffic windows
- Enforce resource limits and quotas (e.g., CPU/memory)
- Isolate workloads (separate nodes, namespaces)
- Use priority/QoS and rate limiting
- Monitor and alert on resource contention early

Incident 45 — The Alert Storm That Hid the Critical Failure

- Hundreds of secondary alerts obscure the initiating incident.
- Investigate alert grouping, dependency mapping, inhibition, and escalation.
- Lab: Build an alert-correlation exercise and identify the primary failure.



Key Checks

- Review alert volume, patterns, and timing
- Map service dependencies and blast radius
- Correlate alerts to identify the initiating event
- Check alert rules, thresholds, and duplicate alerts
- Validate escalation path and response time

Fix / Best Practice

- Implement alert grouping and deduplication
- Use dependency-aware alerting (root cause first)
- Apply alert inhibition (suppress downstream noise)
- Set clear escalation and on-call routing
- Continuously tune alert rules based on real incidents

Senior Project: Build a production early-warning system that detects leading indicators, groups symptoms, and reduces alert noise.

The Incident Review That Exposed Organizational Risk

Incident 46 — The Emergency Fix That Was Never Reconciled

- A temporary production workaround becomes a permanent, undocumented dependency.
- Investigate change records, configuration drift, ownership, and expiration controls.
- Lab: Create a post-incident reconciliation workflow.

Incident (Production Failure) → Emergency Fix (Temporary (Workaround) → Runs in Production (Longer Than (Intended) → Undocumented Dependency (Increased Risk)

Key Checks

- Review change records and approvals
- Identify and document the temporary fix
- Check for configuration drift (actual vs. intended)
- Verify ownership and expiration (sunset) plan
- Confirm the fix is reconciled or removed

Fix / Best Practice

- Document all emergency changes
- Set an expiry date and owner for workarounds
- Track drift with configuration and IaC tools
- Plan and execute a permanent, tested fix
- Review and close the change with evidence

Incident 47 — The Shared Service That Had No Clear Owner

- Several teams depend on a critical service, but none owns its recovery.
- Examine service catalogs, escalation paths, runbooks, and operational contracts.
- Lab: Build a service ownership and dependency map.

Multiple Teams (Use the Service) → Shared Service (Critical) → No Clear Owner (Recovery Uncertain) → Prolonged Outage (Business Impact)

Key Checks

- Identify all dependent teams and workloads
- Check the service catalog and ownership
- Review escalation paths and on-call coverage
- Validate runbooks and recovery procedures
- Confirm RTO/RPO and support commitments

Fix / Best Practice

- Assign a single, accountable service owner
- Maintain an up-to-date service catalog
- Define and test escalation and on-call paths
- Create and validate a runbook with recovery steps
- Set and agree on operational contracts (SLOs/SLAs)

Incident 48 — The Postmortem That Fixed the Symptom but Preserved the Risk

- Corrective actions focus on restarts and dashboards instead of architectural failure modes.
- Examine contributing factors, action effectiveness, recurrence risk, and validation.
- Lab: Rewrite a weak incident review into an evidence-based postmortem.

Incident Review (Findings) → Symptom Fix (Restart / Monitor) → Root Cause Unaddressed (Architecture Risk) → High Risk of Recurrence (Likely to Happen Again)

Key Checks

- Validate the real root cause and contributing factors
- Assess if actions address the failure mode
- Check for evidence and data (logs, metrics, timeline)
- Evaluate recurrence risk and impact
- Confirm a validation plan (tests, monitoring, rollout)

Fix / Best Practice

- Use a structured, evidence-based postmortem
- Address the architectural or design root cause
- Define measurable and testable corrective actions
- Implement and verify changes (rollback if needed)
- Track and review recurrence over time

Senior Project: Build an incident management package with ownership, timelines, evidence, corrective actions, and recurrence tests.

Final Practical Assessment: A Multi-System Production Outage

Incident 49 — The Recovery Attempt That Triggered a Second Outage

A payment platform suffers intermittent database latency. Engineers increase application timeouts and scale out workers.

Database connections multiply, queue consumers retry aggressively, and the surviving Availability Zone becomes saturated.

Database Latency (Initial Fault) → Timeouts + Scale Out (Secondary Actions) → Queue Retry Storm (Amplifies Load) → AZ Saturated (Second Outage)

The Learner Must:

- Identify the original fault and secondary failures.
- Correlate metrics, logs, traces, deployment history, and database evidence.
- Choose the safest mitigation sequence.
- Preserve payment correctness and prevent duplicate processing.
- Restore service without triggering another overload.

Key Areas to Investigate

- Application, database, queue, and infrastructure metrics.
- Application and database logs.
- Distributed traces and request flows.
- Deployment and change history.
- Resource usage and scaling events.
- Reproduction steps and validation tests.

Incident 50 — The Production System That Passed Every Individual Health Check

A simulated enterprise application uses AWS, EKS, RDS, Redis, SQS, Nginx, Terraform, and GitHub Actions. Each component reports healthy, but customers experience failed payments, delayed orders, and inconsistent account information.

AWS (Infrastructure) → EKS (Kubernetes) → RDS (Database) → Redis (Cache) → SQS (Queue) → Nginx (Proxy) → Terraform (IaC) → GitHub Actions (CI/CD)

The Learner Must:

- Build a complete dependency and failure-domain map.
- Determine which failures are independent and which are cascading.
- Establish incident command and communication.
- Recover the system while preserving data integrity.
- Produce a root-cause analysis with verified evidence.
- Implement corrective changes and prove they prevent recurrence.

Key Areas to Investigate

- Service health vs. real user experience.
- Cross-service dependencies.
- Failure domains and blast radius.
- Logs, metrics, and traces across all services.
- Configuration, limits, and resource usage.
- Data consistency and integrity checks.
- Recovery validation and monitoring.

Final Capstone Project: Deliver a production incident response portfolio with a reproducible incident environment, investigation commands, captured evidence, remediation code, dashboards, recovery tests, and an executive incident report.